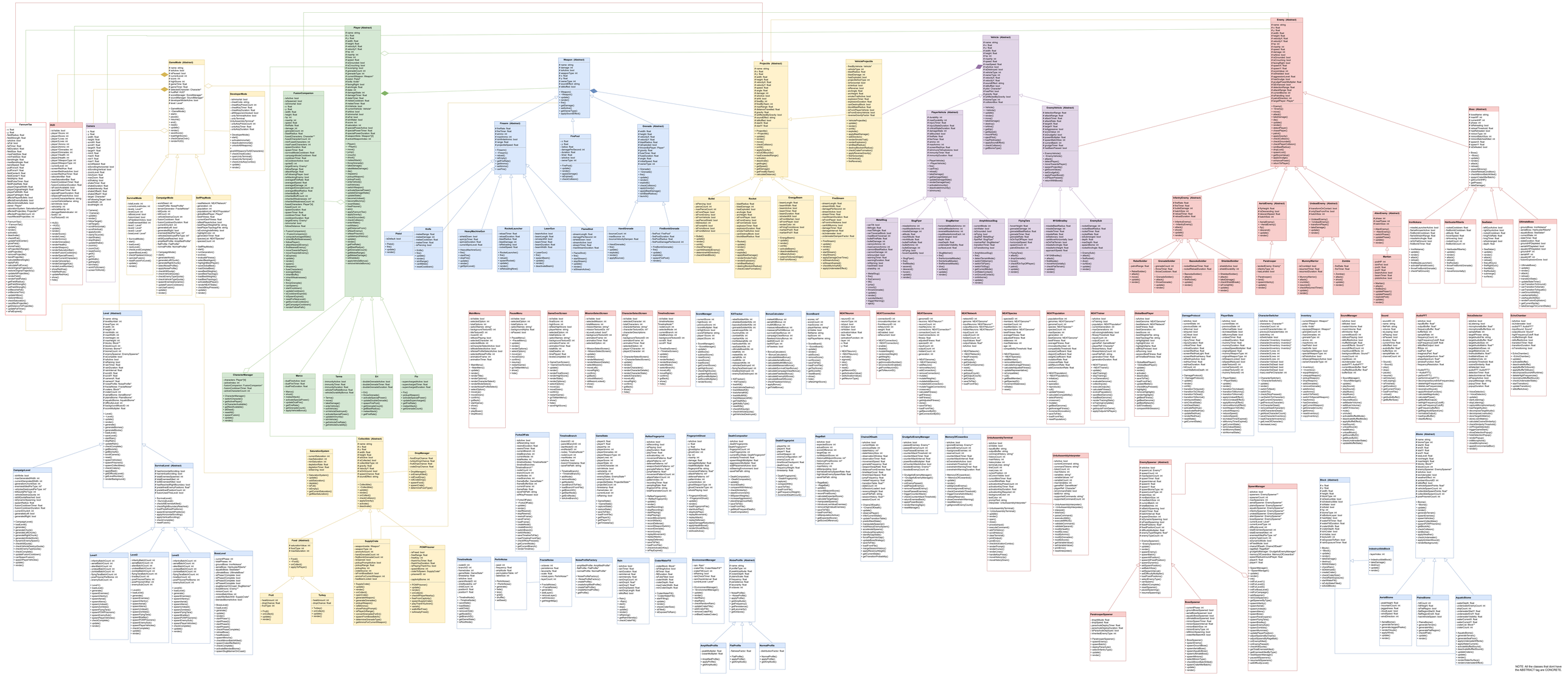

UML Design for Metal Slug

Final Project of OOP Spring 2026

Prepared By:
Laiba Naveed (25I-0818)
Bhavish Jan (25I-0990)

FannumTax

- x: float
- y: float
- fieldRadius: float
- fieldStrength: float
- isActive: bool
- isFat: bool
- fatTimer: float
- fatDuration: float
- fieldSize: float
- maxFieldSize: float
- minFieldSize: float
- bendAngle: float
- maxBendAngle: float
- bendSpeed: float
- pullForceX: float
- pullForceY: float
- fieldCenterX: float
- fieldCenterY: float
- fieldAlpha: float
- fieldPulseTimer: float
- fieldPulseRate: float
- playerOriginalWidth: float
- playerOriginalHeight: float
- playerFatWidth: float
- playerFatHeight: float
- affectsPlayerBullets: bool
- affectsEnemyBullets: bool
- affectsVehicleBullets: bool
- owner: Player*
- saturationSystem: SaturationSystem*
- affectedProjectiles: Projectile**
- affectedProjectileCount: int
- maxAffectedProjectiles: int

+ FannumTax()
+ ~FannumTax()
+ update()
+ render()
+ activate()
+ deactivate()
+ checkIfFat()
+ updateFieldCenter()
+ growField()
+ shrinkField()
+ isProjectileInRange()
+ addProjectile()
+ removeProjectile()
+ bendProjectile()
+ calculateBendAngle()
+ applyPull()
+ calculatePullForce()
+ storeOriginalTrajectory()
+ restoreOriginalTrajectory()
+ updateAllProjectiles()
+ renderField()
+ renderFieldPulse()
+ getFieldRadius()
+ getFieldStrength()
+ setFieldStrength()
+ onBecomeFat()
+ onBecomeThin()
+ updateHitbox()
+ restoreHitbox()
+ checkSaturation()
+ canAffectProjectile()
+ getDistanceToProjectile()
+ updateFatTimer()
+ isFatExpired()

HUD

- isVisible: bool
- player1Score: int
- player2Score: int
- player1Lives: int
- player2Lives: int
- player1Ammo: int
- player2Ammo: int
- player1Grenades: int
- player2Grenades: int
- player1Health: int
- player2Health: int
- player1WeaponType: int
- player2WeaponType: int
- currentLevel: int
- screenRedHue: float
- screenRedHueActive: bool
- screenRedHueTimer: float
- saturationBar: float
- maxSaturationBar: float
- fusionCooldownTimer: float
- fusionCooldownDuration: float
- isFusionAvailable: bool
- specialPowerTimer: float
- specialPowerDuration: float
- isSpecialPowerActive: bool
- currentCharacterName: string
- currentVehicleName: string
- isInVehicle: bool
- vehicleHp: int
- vehicleMaxHp: int
- damageStateIndicator: int
- fontID: int
- hudTextureID: int

+ HUD()
+ ~HUD()
+ update()
+ render()
+ renderScore()
+ renderLives()
+ renderAmmo()
+ renderGrenades()
+ renderHealth()
+ renderWeapon()
+ renderSaturationBar()
+ renderFusionCooldown()
+ renderSpecialPower()
+ renderCurrentCharacter()
+ renderVehicleHp()
+ renderDamageHue()
+ renderLevelNumber()
+ showRedHue()
+ hideRedHue()
+ updateRedHue()
+ show()
+ hide()

Camera

- x: float
- y: float
- width: float
- height: float
- scrollX: float
- scrollY: float
- targetX: float
- targetY: float
- minX: float
- maxX: float
- minY: float
- maxY: float
- scrollSpeed: float
- isScrollingHorizontal: bool
- isScrollingVertical: bool
- zoomLevel: float
- minZoom: float
- maxZoom: float
- isShaking: bool
- shakeTimer: float
- shakeDuration: float
- shakeIntensity: float
- shakeOffsetX: float
- shakeOffsetY: float
- target: Character*
- isFollowingTarget: bool
- levelWidth: int
- levelHeight: int

+ Camera()
+ ~Camera()
+ update()
+ render()
+ followTarget()
+ setTarget()
+ scrollHorizontal()
+ scrollVertical()
+ applyScroll()
+ clampToLevel()
+ startShake()
+ stopShake()
+ updateShake()
+ zoomIn()
+ zoomOut()
+ resetZoom()
+ getX()
+ getY()
+ getWidth()
+ getHeight()
+ isInView()
+ worldToScreen()
+ screenToWorld()

***GameMode* (Abstract)**

# name: string
# isActive: bool
# isPaused: bool
# currentLevel: int
# score: int
# highScore: int
# gameTime: float
# gameTimer: float
# selectedCharacter: Character*
# hudRef: HUD*
# scoreManager: ScoreManager*
# soundManager: SoundManager*
# developerModeActive: bool
# level: Level*

+ GameMode()
+ ~GameMode()
+ start()
+ pause()
+ resume()
+ end()
+ reset()
+ update()
+ render()
+ saveScore()
+ loadHighScore()
+ checkGameOver()
+ renderHUD()

DeveloperMode

- isImmortal: bool
- cheatCode: string
- cheatKeyPressCount: int
- cheatKeyTimer: float
- cheatKeyDuration: float
- allWeaponsUnlocked: bool
- urduTerminalActive: bool
- urduTerminal: UrduAssemblyTerminal*
- urduKeyPressCount: int
- urduKeyTimer: float
- urduKeyDuration: float

+ DeveloperMode()
+ start()
+ activateImmortal()
+ deactivateImmortal()
+ unlockAllWeapons()
+ giveAllWeaponsToAllCharacters()
+ checkCheatCode()
+ openUrduTerminal()
+ closeUrduTerminal()
+ checkUrduKeyCombo()
+ update()
+ render()

SurvivalMode

- totalLevels: int
- currentLevelIndex: int
- levels: Level**
- levelCount: int
- isBossLevel: bool
- fusionUsed: bool
- isFlawlessVictory: bool
- totalEnemiesKilled: int
- level1: Level*
- level2: Level*
- level3: Level*
- bossLevel: Level*

+ SurvivalMode()
+ start()
+ loadLevel()
+ nextLevel()
+ checkLevelComplete()
+ spawnBossLevel()
+ checkFlawlessVictory()
+ canUseFusion()
+ resetFusionForLevel()
+ update()
+ render()

CampaignMode

- worldSeed: int
- noiseProfile: NoiseProfile*
- terrainGenerator: FractalNoise*
- killQuota: int
- killCount: int
- vehicleDestroyCount: int
- fusionCooldown: float
- fusionCooldownDuration: float
- currentChunkX: int
- generatedLeft: bool
- generatedRight: bool
- enemiesKilledPerType: int*
- vehiclesDestroyedPerType: int*
- isKillQuotaReached: bool
- amplifiedProfile: AmplifiedProfile*
- flatProfile: FlatProfile*
- normalProfile: NormalProfile*

+ CampaignMode()
+ start()
+ generateWorld()
+ generateLeftChunk()
+ generateRightChunk()
+ selectNoiseProfile()
+ applyNoiseProfile()
+ checkKillQuota()
+ checkVehicleQuota()
+ checkEnemyTypeQuota()
+ checkVehicleTypeQuota()
+ spawnEnemiesDynamic()
+ updateFusionCooldown()
+ canUseFusion()
+ update()
+ render()

SelfPlayMode

- neatNetwork: NEATNetwork*
- generation: int
- population: int
- populationList: NEATPopulation*
- globalBestPlayer: Player*
- bestFitness: float
- currentGenFitness: float
- isBestPlayerActive: bool
- bestPlayerWeightsFile: string
- bestPlayerTopologyFile: string
- isEvolvingIndefinitely: bool
- highlightColor: int
- generationTimer: float
- speciesList: NEATSpecies*
- speciesCount: int

+ SelfPlayMode()
+ start()
+ runGeneration()
+ evolve()
+ evaluateFitness()
+ selectBestAgent()
+ highlightBestPlayer()
+ saveGlobalBest()
+ loadGlobalBest()
+ saveBestWeights()
+ saveBestTopology()
+ loadBestWeights()
+ loadBestTopology()
+ activateBestPlayer()
+ renderNEATStats()
+ checkBKeyPressed()
+ update()
+ render()

FusionCompanion

- isActive: bool
- isSpawned: bool
- isImmortal: bool
- x: float
- y: float
- width: float
- height: float
- hp: int
- maxHp: int
- speed: float
- fireRate: float
- damage: int
- grenadeCount: int
- blastRadius: float
- fusedCharacters: Character**
- fusedCharacterCount: int
- minFusedCharacters: int
- maxFusedCharacters: int
- spawnDuration: float
- spawnTimer: float
- survivalModeCooldown: float
- campaignModeCooldown: float
- cooldownTimer: float
- isCooldownActive: bool
- spriteID: int
- targetEnemy: Enemy*
- followRange: float
- attackRange: float
- isFollowingPlayer: bool
- isAttackingEnemy: bool
- averagedFireRate: float
- averagedSpeed: float
- averagedDamage: int
- averagedGrenadeCount: int
- averagedBlastRadius: float
- inheritedBuffs: int*
- inheritedBuffCount: int
- inheritedWeaknesses: int*
- inheritedWeaknessCount: int
- fusedCharacters: Player*[4]
- fusedCount: int
- spawnDuration: float
- spawnTimer: float
- cooldownTimer: float
- cooldownDuration: float
- targetEnemy: Enemy*
- isImmortal: bool
- followDistance: float

+ FusionCompanion()
+ ~FusionCompanion()
+ computeAveragedStats()
+ inheritBuffsWeaknesses()
+ followPlayer()
+ attackNearestEnemy()
+ findNearestEnemy()
+ moveTowardsPlayer()
+ moveTowardsEnemy()
+ isExpired()
+ update()
+ render()
+ spawn()
+ despawn()
+ fuseCharacters()
+ averageStats()
+ inheritBuffs()
+ inheritWeaknesses()
+ fire()
+ throwGrenade()
+ canSpawn()
+ startCooldown()
+ updateCooldown()
+ isCooldownExpired()
+ isSpawnExpired()
+ resetForNewLevel()
+ getSurvivalCooldown()
+ getCampaignCooldown()
+ renderFollowPath()

Player (Abstract)

# name: string
# x: float
# y: float
# width: float
# height: float
# velocityX: float
# velocityY: float
# hp: int
# maxHp: int
# lives: int
# speed: float
# isGrounded: bool
# isCrouching: bool
# isJumping: bool
# grenadeCount: int
# grenadeType: int
# currentWeapon: Weapon*
# pistol: Pistol*
# knife: Knife*
# facingRight: bool
# aimAngle: float
# state: int
# damageState: int
# damageTimer: float
# stateTimer: float
# meleeCooldown: float
# meleeTimer: float
# inVehicle: bool
# currentVehicle: Vehicle*
# isAlive: bool
# isImmortal: bool
# isFat: bool
# isInWater: bool
# score: int
# saturation: int
# specialPowerActive: bool
# specialPowerTimer: float
# specialPowerDuration: float
# weaponSlots: Weapon*[2]
# activeWeaponIndex: int

+ Player()
+ ~Player()
+ move()
+ jump()
+ crouch()
+ shoot()
+ meleeAttack()
+ throwGrenade()
+ takeDamage()
+ takeMeleeDamage()
+ die()
+ respawn()
+ pickupWeapon()
+ pickupFood()
+ enterVehicle()
+ exitVehicle()
+ switchWeapon()
+ activateSpecialPower()
+ updateDamageState()
+ updateState()
+ becomeUndead()
+ becomeMummy()
+ revertState()
+ freePrisoner()
+ aim()
+ applyFannumTax()
+ applyGravity()
+ checkGrounded()
+ checkCollision()
+ enterWater()
+ exitWater()
+ getDistanceFrom()
+ isNearEnemy()
+ isNearPrisoner()
+ updateInputHistory()
+ update()
+ render()
+ getFireRate()
+ getBlastRadius()
+ getGrenadeCount()
+ getMeleeRange()
+ getMeleeDamage()
+ isShielded()
+ applyVehicleBonus()
+ removeVehicleBonus()

Weapon (Abstract)

# name: string
# damage: int
# isActive: bool
# weaponType: int
# x: float
# y: float
# ownerType: int
# soundEffect: string
# isMuffled: bool

+ Weapon()
+ ~Weapon()
+ update()
+ render()
+ fire()
+ getDamage()
+ setActive()
+ getOwnerType()
+ applySoundEffect()

Firearm (Abstract)

# fireRate: float
# fireTimer: float
# ammo: int
# maxAmmo: int
# isInfiniteAmmo: bool
# range: float
# projectileSpeed: float

+ Firearm()
+ ~Firearm()
+ reload()
+ isEmpty()
+ getFireRate()
+ getAmmo()
+ setAmmo()
+ canFire()

FirePool

- x: float
- y: float
- radius: float
- damagePerSecond: int
- duration: float
- timer: float
- isActive: bool
- ownerType: int

+ FirePool()
+ update()
+ render()
+ applyDamage()
+ isExpired()
+ checkCollision()

Grenade (Abstract)

# width: float
# height: float
# velocityX: float
# velocityY: float
# blastRadius: float
# isExploded: bool
# thrownByPlayer: Player*
# gravity: float
# fuseTimer: float
# fuseDuration: float
# angle: float
# initialSpeed: float
# ownerType: int

+ Grenade()
+ ~Grenade()
+ fire()
+ update()
+ render()
+ explode()
+ checkCollision()
+ applyGravity()
+ applyBlastDamage()
+ isInBlastRadius()
+ launch()

Pistol

- isDefault: bool

+ Pistol()
+ fire()
+ update()
+ render()

Knife

- meleeRange: float
- meleeDamage: int
- meleeCooldown: float
- meleeTimer: float
- isPiercing: bool

+ Knife()
+ fire()
+ update()
+ render()
+ isInRange()
+ canMelee()
+ resetCooldown()

HeavyMachineGun

- isHeldDown: bool
- spinUpTimer: float
- spinUpDuration: float
- currentSpinLevel: float

+ HeavyMachineGun()
+ fire()
+ startFire()
+ stopFire()
+ update()
+ render()
+ getSpinLevel()

RocketLauncher

- reloadTimer: float
- reloadDuration: float
- blastRadius: float
- blastDamage: int
- isReloading: bool
- rocketSpeed: float

+ RocketLauncher()
+ fire()
+ reload()
+ spawnRocket()
+ update()
+ render()
+ isReloadingNow()

LaserGun

- beamActive: bool
- beamLength: float
- isInstantKill: bool
- beamTimer: float
- beamDuration: float
- beamWidth: float

+ LaserGun()
+ fire()
+ spawnBeam()
+ update()
+ render()
+ deactivateBeam()

FlameShot

- streamLength: float
- damagePerSecond: int
- isHeldDown: bool
- streamActive: bool
- streamTimer: float
- maxStreamDuration: float

+ FlameShot()
+ fire()
+ startStream()
+ stopStream()
+ update()
+ render()
+ isStreamActive()

HandGrenade

- bounceCount: int
- maxBounce: int
- bounceVelocityDampen: float

+ HandGrenade()
+ fire()
+ update()
+ explode()
+ render()
+ applyBounce()

FireBombGrenade

- firePool: FirePool*
- firePoolDuration: float
- firePoolRadius: float
- firePoolDamagePerSecond: int

+ FireBombGrenade()
+ fire()
+ update()
+ explode()
+ spawnFirePool()
+ render()

Projectile (Abstract)

# name: string
# x: float
# y: float
# width: float
# height: float
# velocityX: float
# velocityY: float
# speed: float
# angle: float
# damage: int
# isActive: bool
# isHit: bool
# firedBy: int
# firedByTeam: int
# maxRange: float
# distanceTravelled: float
# gravity: float
# isAffectedByGravity: bool
# soundEffect: string
# isMuffled: bool
# startX: float
# startY: float

+ Projectile()
+ ~Projectile()
+ update()
+ render()
+ move()
+ checkCollision()
+ onHit()
+ applyGravity()
+ isOutOfBounds()
+ hasExceededRange()
+ activate()
+ deactivate()
+ getAngle()
+ getSpeed()
+ getDamage()
+ getFiredByTeam()
+ calculateDistance()

VehicleProjectile

- firedByVehicle: Vehicle*
- vehicleType: int
- blastRadius: float
- blastDamage: int
- hasExploded: bool
- projectileSubType: int
- isHorizontal: bool
- isVertical: bool
- isReverse: bool
- arcAngle: float
- arcHeight: float
- smokeTrailActive: bool
- explosionTimer: float
- explosionDuration: float
- canDestroyBlock: bool
- blockBlastRadius: float
- isFromPlayerVehicle: bool
- isFromEnemyVehicle: bool
- reverseGravityFactor: float

+ VehicleProjectile()
+ update()
+ render()
+ onHit()
+ explode()
+ applyBlastDamage()
+ setDirection()
+ renderSmokeTrail()
+ renderExplosion()
+ isInBlastRadius()
+ destroyBlocksInRadius()
+ checkCraterFormation()
+ applyReverseGravity()
+ fireHorizontal()
+ fireVertical()
+ fireReverse()

Bullet

- isPiercing: bool
- pierceCount: int
- maxPierceCount: int
- isFromPlayer: bool
- isFromEnemy: bool
- isFromVehicle: bool
- canPierceShield: bool
- hitDirection: int
- isFrontalHit: bool
- isFromAbove: bool
- isFromBehind: bool

+ Bullet()
+ update()
+ render()
+ onHit()
+ checkPiercing()
+ canHitShieldedEnemy()
+ determineHitDirection()
+ checkShieldBlock()

Rocket

- blastRadius: float
- blastDamage: int
- hasExploded: bool
- arcAngle: float
- arcHeight: float
- isFromPlayer: bool
- isFromEnemy: bool
- isFromVehicle: bool
- explosionTimer: float
- explosionDuration: float
- smokeTrailActive: bool
- canDestroyBlock: bool
- blockBlastRadius: float

+ Rocket()
+ update()
+ render()
+ onHit()
+ explode()
+ applyBlastDamage()
+ renderSmokeTrail()
+ renderExplosion()
+ isInBlastRadius()
+ destroyBlocksInRadius()
+ checkCraterFormation()

EnergyBeam

- beamLength: float
- beamWidth: float
- beamActive: bool
- beamTimer: float
- beamDuration: float
- isInstantKill: bool
- isFromMartian: bool
- isFromPlayer: bool
- renderColor: int
- isFiringFromAbove: bool
- martianPodX: float
- martianPodY: float

+ EnergyBeam()
+ update()
+ render()
+ onHit()
+ activateBeam()
+ deactivateBeam()
+ isBeamActive()
+ extendToScreenEdge()
+ fireFromAbove()

FireStream

- streamLength: float
- streamWidth: float
- damagePerSecond: int
- streamActive: bool
- streamTimer: float
- maxStreamDuration: float
- isFromIronNokana: bool
- canKillMummy: bool
- damageTimer: float
- damageInterval: float
- isUnderwater: bool
- underwaterDampenFactor: float

+ FireStream()
+ update()
+ render()
+ onHit()
+ startStream()
+ stopStream()
+ applyDamageOverTime()
+ isStreamActive()
+ isStreamExpired()
+ checkMummyKill()
+ applyUnderwaterEffect()

Vehicle (Abstract)

# name: string
# x: float
# y: float
# width: float
# height: float
# hp: int
# maxHp: int
# speed: float
# maxSpeed: float
# isActive: bool
# isDestroyed: bool
# vehicleType: int
# ownerType: int
# velocityX: float
# velocityY: float
# soundEffect: string
# isMuffled: bool
# pilot: Character*
# hasPilot: bool
# gravity: float
# isAffectedByGravity: bool
# biomeType: int
# collisionBox: float

+ Vehicle()
+ ~Vehicle()
+ update()
+ render()
+ move()
+ takeDamage()
+ destroy()
+ isAlive()
+ getHp()
+ getSpeed()
+ mountPilot()
+ ejectPilot()
+ hasPilotInside()
+ applySoundEffect()
+ checkCollision()
+ getBiomeType()

PlayerVehicle (Abstract)

# durability: int
# maxDurability: int
# injuryTimer: float
# injuryStateDuration: float
# criticalStateDuration: float
# damageState: int
# isMounted: bool
# fireRate: float
# fireTimer: float
# ammo: int
# maxAmmo: int
# screenRedHue: float
# isImmuneToExplosions: bool
# immunityTimer: float
# immunityDuration: float

+ PlayerVehicle()
+ ~PlayerVehicle()
+ fire()
+ canFire()
+ reload()
+ takeDamage()
+ getDamageState()
+ updateDamageState()
+ renderDamageHue()
+ activateImmunity()
+ deactivateImmunity()
+ isImmune()

EnemyVehicle (Abstract)

# detectionRange: float
# attackRange: float
# attackTimer: float
# attackRate: float
# targetX: float
# targetY: float
# isAggressive: bool
# scoreValue: int
# isGrudgeful: bool
# powerMultiplier: float
# spawnBatchSize: int
# currentBatch: int
# grudgeTimer: float
# hasBeenPassed: bool

+ EnemyVehicle()
+ ~EnemyVehicle()
+ attack()
+ detectPlayer()
+ moveTowardsPlayer()
+ spawnProjectile()
+ getScoreValue()
+ setGrudgeful()
+ applyPowerBoost()
+ onPlayerPassed()
+ returnWithBoost()

MetalSlug

- canTilt: bool
- tiltAngle: float
- maxTiltAngle: float
- canTraverseBlock: bool
- maxBlockTraversal: int
- bulletFireRate: float
- bulletDamage: int
- cannonAmmo: int
- maxCannonAmmo: int
- cannonBlastRadius: float
- cannonDamage: int
- isGrounded: bool
- warningTimer: float
- warningDuration: float
- isWarning: bool
- hasShield: bool
- shieldHp: int

+ MetalSlug()
+ fire()
+ fireCannon()
+ tilt()
+ jump()
+ crouch()
+ throwGrenade()
+ update()
+ render()
+ suicideAttack()
+ triggerWarning()
+ eject()

SlugFlyer

- missileAmmo: int
- maxMissileAmmo: int
- missileDamage: int
- altitude: float
- maxAltitude: float
- minAltitude: float
- bulletFireRate: float
- bulletAmmo: int
- bulletDamage: int
- isFlying: bool
- hoverCapability: bool

+ SlugFlyer()
+ fire()
+ fireMissile()
+ ascend()
+ descend()
+ hover()
+ update()
+ render()

SlugMariner

- horizontalMissileAmmo: int
- verticalMissileAmmo: int
- reverseMissileAmmo: int
- maxMissileAmmoEach: int
- isSubmerged: bool
- depth: float
- maxDepth: float
- underwaterVisibility: float
- surfaceLevel: float

+ SlugMariner()
+ fire()
+ fireHorizontalMissile()
+ fireVerticalMissile()
+ fireReverseMissile()
+ dive()
+ surface()
+ update()
+ render()

AmphibiousSlug

- currentMode: int
- isInWater: bool
- isGrounded: bool
- isFlying: bool
- slugRef: MetalSlug*
- flyerRef: SlugFlyer*
- marinerRef: SlugMariner*
- transitionTimer: float
- isTransitioning: bool

+ AmphibiousSlug()
+ fire()
+ detectTerrainMode()
+ switchToMariner()
+ switchToFlyer()
+ switchToSlug()
+ getCurrentMode()
+ onWaterContact()
+ onGroundContact()
+ onAirborne()
+ update()
+ render()

FlyingTara

- hoverHeight: float
- grenadeDamage: int
- grenadeBlastRadius: float
- grenadeTimer: float
- grenadeRate: float
- isOnTopOfPlayer: bool
- horizontalSpeed: float
- spawnCount: int
- patrolDirection: int

+ FlyingTara()
+ attack()
+ dropGrenade()
+ hover()
+ checkIfOnTopOfPlayer()
+ patrol()
+ update()
+ render()

M15ABradley

- missileBlastRadius: float
- missileDamage: int
- missileTimer: float
- missileRate: float
- isStationary: bool
- canMoveHorizontally: bool
- isOnFlatTerrain: bool
- missileArcAngle: float
- missileArcHeight: float
- approachSpeed: float

+ M15ABradley()
+ attack()
+ fireMissile()
+ moveHorizontally()
+ checkFlatTerrain()
+ update()
+ render()

EnemySub

- rocketDamage: int
- rocketBlastRadius: float
- rocketTimer: float
- rocketRate: float
- isSubmerged: bool
- depth: float
- maxDepth: float
- spawnBiome: int
- rocketArcAngle: float

+ EnemySub()
+ attack()
+ fireRocket()
+ dive()
+ surface()
+ update()
+ render()

Enemy (Abstract)

# name: string
# x: float
# y: float
# width: float
# height: float
# velocityX: float
# velocityY: float
# hp: int
# maxHp: int
# speed: float
# damage: int
# isAlive: bool
# isGrounded: bool
# isCrouching: bool
# facingRight: bool
# spawnX: float
# spawnY: float
# scoreValue: int
# isShielded: bool
# aggressionLevel: float
# hasGrudge: bool
# grudgePowerMultiplier: float
# isEnhanced: bool
# detectionRange: float
# attackRange: float
# currentBiome: int
# isPatrolling: bool
# patrolDirection: int
# targetPlayer: Player*

+ Enemy()
+ ~Enemy()
+ move()
+ attack()
+ takeDamage()
+ die()
+ update()
+ render()
+ detectPlayer()
+ chasePlayer()
+ patrol()
+ applyGravity()
+ checkCollision()
+ checkGrounded()
+ checkPlayerCollision()
+ isInBlastRadius()
+ dropLoot()
+ spawnLoot()
+ getScoreValue()
+ applyGrudge()
+ enhancePower()
+ returnToPlayer()

***Boss* (Abstract)**

# bossName: string
# maxHP: int
# currentHP: int
# phase: int
# isRetreating: bool
# retreatThreshold: float
# hasRetreated: bool
# minionType: int
# minionBatchSize: int
# minionBatchesKilled: int
# spawnX: float
# spawnY: float
# isDefeated: bool

+ Boss()
+ ~Boss()
+ update()
+ render()
+ attack()
+ retreat()
+ spawnMinions()
+ checkRetreatCondition()
+ checkMinionBatchKilled()
+ spawnCrateAfterBatch()
+ getCurrentHP()
+ getPhase()
+ takeDamage()

InfantryEnemy (Abstract)

# fireRate: float
# fireTimer: float
# bulletDamage: int
# batchSize: int
# reloadTimer: float
# reloadDuration: float

+ InfantryEnemy()
+ ~InfantryEnemy()
+ attack()
+ shoot()
+ update()
+ render()

AerialEnemy (Abstract)

# flyHeight: float
# isDescending: bool
# descentSpeed: float
# batchSize: int

+ AerialEnemy()
+ ~AerialEnemy()
+ fly()
+ descend()
+ update()
+ render()

UndeadEnemy (Abstract)

# transformOnContact: bool
# onlyDiesFromFire: bool
# batchSize: int

+ UndeadEnemy()
+ ~UndeadEnemy()
+ transformPlayer()
+ takeDamage()
+ update()
+ render()

AlienEnemy (Abstract)

# phase: int
# maxPhase: int
# batchSize: int

+ AlienEnemy()
+ ~AlienEnemy()
+ switchPhase()
+ updatePhase()
+ update()
+ render()

IronNokana

- missileLauncherActive: bool
- flamethrowerActive: bool
- fireBombCooldown: float
- flameStreamRange: float
- missileArcAngle: float
- isOnFlatGround: bool
- fireBombTimer: float

+ IronNokana()
+ update()
+ render()
+ attack()
+ retreat()
+ fireMissileLauncher()
+ activateFlamethrower()
+ throwFireBombGrenade()
+ checkFlatGround()

HairbusterRiberts

- rocketCooldown: float
- fireBombCooldown: float
- altitude: float
- hoverSpeed: float
- isHovering: bool
- rocketCount: int
- fireBombCount: int

+ HairbusterRiberts()
+ update()
+ render()
+ attack()
+ retreat()
+ fireRocket()
+ throwFireBombGrenade()
+ hover()
+ moveHorizontally()

SeaSatan

- leftGunActive: bool
- rightGunActive: bool
- rocketCooldown: float
- hmgFireRate: float
- hmgTimer: float
- isSubmerged: bool
- depth: float

+ SeaSatan()
+ update()
+ render()
+ attack()
+ retreat()
+ fireLeftGun()
+ fireRightGun()
+ fireHMG()
+ fireRocket()
+ submerge()
+ surface()

UltimateBoss

- groundBoss: IronNokana*
- aerialBoss: HairbusterRiberts*
- aquaticBoss: SeaSatan*
- currentState: int
- stateTimer: float
- stateDuration: float
- groundHPDepleted: bool
- aerialHPDepleted: bool
- aquaticHPDepleted: bool
- groundHP: int
- aerialHP: int
- aquaticHP: int
- fusionExplosionDone: bool

+ UltimateBoss()
+ update()
+ render()
+ attack()
+ retreat()
+ transitionState()
+ updateStateTimer()
+ canTransitionToGround()
+ canTransitionToAerial()
+ canTransitionToAquatic()
+ useGroundAbility()
+ useAerialAbility()
+ useAquaticAbility()
+ renderFusionExplosion()
+ getCurrentSprite()
+ takeDamageToCurrentState()

RebelSoldier

- patrolRange: float

+ RebelSoldier()
+ attack()
+ move()
+ update()
+ render()

GrenadeSoldier

- grenadeCount: int
- throwTimer: float
- throwCooldown: float

+ GrenadeSoldier()
+ attack()
+ throwGrenade()
+ update()
+ render()

BazookaSoldier

- rocketReloadTimer: float
- rocketReloadDuration: float

+ BazookaSoldier()
+ attack()
+ fireRocket()
+ update()
+ render()

ShieldedSoldier

- shieldActive: bool
- shieldDurability: int

+ ShieldedSoldier()
+ attack()
+ takeDamage()
+ checkShieldBreak()
+ isFrontalHit()
+ update()
+ render()

Paratrooper

- landedEnemy: Enemy*
- infantryType: int
- hasLanded: bool

+ Paratrooper()
+ land()
+ spawnInfantry()
+ update()
+ render()

MummyWarrior

- isCrumbled: bool
- resurrectTimer: float
- resurrectDuration: float

+ MummyWarrior()
+ attack()
+ crumble()
+ resurrect()
+ checkResurrectTimer()
+ update()
+ render()

Zombie

- fireRate: float
- fireTimer: float

+ Zombie()
+ attack()
+ shoot()
+ update()
+ render()

Martian

- podHP: int
- isInPod: bool
- podX: float
- podY: float
- beamActive: bool
- beamTimer: float
- pistolActive: bool

+ Martian()
+ attack()
+ fireBeam()
+ updatePhase1()
+ updatePhase2()
+ explodePod()
+ update()
+ render()

CharacterManager

- characters: Player*[4]
- activeIndex: int
- fusionCompanion: FusionCompanion*
- fusionCooldownTimer: float
- activeCharacterCount: int

+ CharacterManager()
+ switchCharacter()
+ getActivePlayer()
+ isCharacterAvailable()
+ getNextAvailable()
+ allDead()
+ resetAll()
+ spawnFusion()
+ update()
+ render()

Marco

- dualFireActive: bool
- dualFireTimer: float
- dualFireDuration: float

+ Marco()
+ shoot()
+ meleeAttack()
+ activateSpecialPower()
+ updateDualFire()
+ getFireRate()
+ getGrenadeCount()
+ applyVehicleBonus()

Tarma

- immunityActive: bool
- immunityTimer: float
- immunityDuration: float
- vehicleFireRateBonus: float
- vehicleDurabilityBonus: float

+ Tarma()
+ move()
+ takeDamage()
+ enterVehicle()
+ exitVehicle()
+ onVehicleDestroyed()
+ activateSpecialPower()
+ updateImmunity()
+ isInVehicle()
+ getVehicleFireRate()
+ getVehicleDurability()

Eri

- doubleGrenadeActive: bool
- doubleGrenadeTimer: float
- doubleGrenadeDuration: float

+ Eri()
+ throwGrenade()
+ activateSpecialPower()
+ updateDoubleGrenade()
+ spawnFirePool()
+ getGrenadeCount()
+ getBlastRadius()
+ meleeAttack()
+ getFireRate()

Fio

- superchargedActive: bool
- superchargedTimer: float
- superchargedDuration: float

+ Fio()
+ pickupWeapon()
+ activateSpecialPower()
+ updateSupercharged()
+ getFireRate()
+ meleeAttack()
+ getGrenadeCount()

Level (Abstract)

# name: string
# levelNumber: int
# levelType: int
# width: int
# height: int
# minWidth: int
# maxWidth: int
# minHeight: int
# maxHeight: int
# blocks: Block**
# blockCount: int
# biomes: Biome**
# biomeCount: int
# enemySpawner: EnemySpawner*
# isComplete: bool
# isActive: bool
# isRaining: bool
# rainTimer: float
# rainDuration: float
# rainInterval: float
# scrollX: float
# scrollY: float
# cameraX: float
# cameraY: float
# noiseProfile: NoiseProfile*
# fractalNoise: FractalNoise*
# craters: Block**
# craterCount: int
# aerialBiome: AerialBiome*
# plainsBiome: PlainsBiome*
# aquaticBiome: AquaticBiome*
# difficultyLevel: int
# backgroundMusicID: int
# scoreMultiplier: float

+ Level()
+ ~Level()
+ update()
+ render()
+ generate()
+ generateBiomes()
+ generateBlocks()
+ loadLevel()
+ saveLevel()
+ startRain()
+ stopRain()
+ updateRain()
+ checkComplete()
+ getBlockAt()
+ getBiomeAt()
+ scrollCamera()
+ reset()
+ spawnVehicles()
+ spawnPrisoners()
+ spawnCollectibles()
+ checkCraters()
+ applyBlast()
+ getDifficultyLevel()
+ getLevelNumber()
+ renderBackground()

CampaignLevel

- isInfinite: bool
- maxGeneratedWidth: int
- currentGeneratedWidth: int
- generationChunkSize: int
- enemiesKilledPerType: int*
- vehiclesDestroyedPerType: int*
- killQuotaPerType: int
- vehicleDestroyQuota: int
- isKillQuotaReached: bool
- isDynamicSpawning: bool
- spawnRadius: float
- fusionCooldownTimer: float
- fusionCooldownDuration: float
- currentChunkX: int
- generatedLeft: bool
- generatedRight: bool

+ CampaignLevel()
+ loadLevel()
+ generate()
+ generateChunk()
+ generateLeftChunk()
+ generateRightChunk()
+ generateInfiniteWorld()
+ dynamicEnemySpawn()
+ checkKillQuota()
+ checkVehicleDestroyQuota()
+ checkEnemyTypeQuota()
+ checkComplete()
+ updateFusionCooldown()
+ canSpawnFusion()
+ spawnPlayerVehicles()
+ update()
+ render()

SurvivalLevel (Abstract)

# hasHorizontalScrolling: bool
# hasVerticalScrolling: bool
# totalEnemiesSpawned: int
# totalEnemiesKilled: int
# isAllEnemiesKilled: bool
# hasReachedRightBoundary: bool
# predefinedEnemyPositions: float*
# predefinedPositionCount: int
# fusionUsedThisLevel: bool

+ SurvivalLevel()
+ ~SurvivalLevel()
+ checkAllEnemiesKilled()
+ checkRightBoundaryReached()
+ loadPredefinedPositions()
+ spawnEnemiesAtPositions()
+ applyHorizontalScrolling()
+ applyVerticalScrolling()
+ checkComplete()
+ resetFusion()

Level1

- infantryBatchCount: int
- aerialBatchCount: int
- alienBatchCount: int
- undeadBatchCount: int
- zombieBatchCount: int
- flyingTaraBatchCount: int
- powPrisonerPerBiome: int
- enemySubCount: int

+ Level1()
+ loadLevel()
+ generate()
+ spawnEnemies()
+ spawnInfantry()
+ spawnAerial()
+ spawnAliens()
+ spawnUndead()
+ spawnZombies()
+ spawnFlyingTara()
+ spawnPOWPrisoners()
+ spawnEnemySub()
+ spawnPlayerVehicles()
+ checkComplete()
+ update()
+ render()

Level2

- infantryBatchCount: int
- aerialBatchCount: int
- alienBatchCount: int
- undeadBatchCount: int
- zombieBatchCount: int
- flyingTaraBatchCount: int
- bradleyCount: int
- powPrisonerPlains: int
- powPrisonerOther: int
- enemySubCount: int

+ Level2()
+ loadLevel()
+ generate()
+ spawnEnemies()
+ spawnInfantry()
+ spawnAerial()
+ spawnAliens()
+ spawnUndead()
+ spawnZombies()
+ spawnFlyingTara()
+ spawnBradley()
+ spawnPOWPrisoners()
+ spawnEnemySub()
+ spawnPlayerVehicles()
+ checkComplete()
+ update()
+ render()

Level3

- infantryBatchCount: int
- aerialBatchCount: int
- alienBatchCount: int
- undeadBatchCount: int
- zombieBatchCount: int
- flyingTaraBatchCount: int
- bradleyCount: int
- powPrisonerPerBiome: int
- enemySubCount: int

+ Level3()
+ loadLevel()
+ generate()
+ spawnEnemies()
+ spawnInfantry()
+ spawnAerial()
+ spawnAliens()
+ spawnUndead()
+ spawnZombies()
+ spawnFlyingTara()
+ spawnBradley()
+ spawnPOWPrisoners()
+ spawnEnemySub()
+ spawnPlayerVehicles()
+ checkComplete()
+ update()
+ render()

BossLevel

- currentPhase: int
- totalPhases: int
- groundBoss: IronNokana*
- aerialBoss: HairbusterRiberts*
- aquaticBoss: SeaSatan*
- ultimateBoss: UltimateBoss*
- isPhase1Complete: bool
- isPhase2Complete: bool
- isPhase3Complete: bool
- isPhase4Complete: bool
- slugMarinerOnCoast: SlugMariner*
- bossMinions: Enemy**
- minionCount: int
- minionBatchSize: int
- crateAfterBatchKill: SupplyCrate*
- blendedBiomeActive: bool

+ BossLevel()
+ loadLevel()
+ generate()
+ update()
+ render()
+ startPhase1()
+ startPhase2()
+ startPhase3()
+ startPhase4()
+ checkPhaseComplete()
+ retreatBoss()
+ fuseBosses()
+ spawnMinions()
+ checkMinionBatchKilled()
+ spawnCrateAfterBatch()
+ checkComplete()
+ activateBlendedBiome()
+ spawnSlugMarinerOnCoast()

Collectible (Abstract)

# name: string
# x: float
# y: float
# width: float
# height: float
# isActive: bool
# isCollected: bool
# collectibleType: int
# gravity: float
# velocityY: float
# isOnGround: bool
# spawnChance: float
# soundEffect: string

+ Collectible()
+ ~Collectible()
+ update()
+ render()
+ onCollect()
+ checkCollision()
+ applyGravity()
+ activate()
+ deactivate()

SaturationSystem

- currentSaturation: int
- maxSaturation: int
- minSaturation: int
- depletionRate: float
- depletionTimer: float
- isStarving: bool

+ SaturationSystem()
+ update()
+ addSaturation()
+ deplete()
+ isFull()
+ isEmpty()
+ getCurrentSaturation()
+ getMaxSaturation()

DropManager

- foodDropChance: float
- turkeyDropChance: float
- fruitDropChance: float
- crateDropChance: float

+ DropManager()
+ onEnemyKilled()
+ rollFoodDrop()
+ rollCrateDrop()
+ spawnFood()
+ spawnCrate()
+ determineFoodType()

Food (Abstract)

# saturationValue: int
# foodType: int
# maxSaturation: int

+ Food()
+ ~Food()
+ onCollect()
+ applyToPlayer()

Fruit

- healAmount: int
- dropChance: float
- fruitType: int

+ Fruit()
+ onCollect()
+ update()
+ render()

Turkey

- healAmount: int
- dropChance: float

+ Turkey()
+ onCollect()
+ update()
+ render()

SupplyCrate

- weaponInside: Weapon*
- weaponType: int
- ammoAmount: int
- handGrenadeCount: int
- fireBombGrenadeCount: int
- isOpened: bool
- pickupPromptActive: bool
- pickupRange: float
- pickupKey: int
- crateSource: int
- isFromBossBatch: bool
- ammoForCurrentWeapon: int
- hasBeenLooted: bool

+ SupplyCrate()
+ update()
+ render()
+ onCollect()
+ openCrate()
+ generateWeapon()
+ generateGrenades()
+ pickupWeapon()
+ refillAmmo()
+ showPickupPrompt()
+ isPlayerInRange()
+ convertGrenadesForEri()
+ spawnFromBossBatch()
+ determineGrenadeType()
+ getAmmoForCurrentWeapon()

POWPrisoner

- isFreed: bool
- freeRange: float
- freeKey: int
- thankYouTimer: float
- thankYouDuration: float
- isPlayingThankYou: bool
- spawnBiome: int
- crateToSpawn: SupplyCrate*
- prisonerID: int

- captivityBiome: int

+ POWPrisoner()
+ update()
+ render()
+ onCollect()
+ checkIfPlayerNearby()
+ freeFromCaptivity()
+ spawnSupplyCrate()
+ playThankYouAnim()
+ vanish()
+ walkAfterFree()
+ isAlreadyFreed()

MainMenu

- isVisible: bool
- selectedOption: int
- totalOptions: int
- optionNames: string*
- backgroundTextureID: int
- titleTextureID: int
- musicID: int
- isMusicPlaying: bool
- selectedCharacter: int
- selectedMode: int
- isCharacterSelectActive: bool
- isModeSelectActive: bool
- isNoiseProfileSelectActive: bool
- selectedNoiseProfile: int
- animationFrame: int
- animationTimer: float

+ MainMenu()
+ ~MainMenu()
+ update()
+ render()
+ renderTitle()
+ renderOptions()
+ renderCharacterSelect()
+ renderModeSelect()
+ renderNoiseProfileSelect()
+ selectOption()
+ moveUp()
+ moveDown()
+ confirm()
+ goBack()
+ startGame()
+ show()
+ hide()
+ playMusic()
+ stopMusic()

PauseMenu

- isVisible: bool
- selectedOption: int
- totalOptions: int
- optionNames: string*
- backgroundAlpha: float
- isPaused: bool

+ PauseMenu()
+ ~PauseMenu()
+ update()
+ render()
+ renderOptions()
+ selectOption()
+ moveUp()
+ moveDown()
+ confirm()
+ resume()
+ restartGame()
+ goToMainMenu()
+ show()
+ hide()

GameOverScreen

- isVisible: bool
- finalScore: int
- highScore: int
- isNewHighScore: bool
- playerName: string
- selectedOption: int
- totalOptions: int
- optionNames: string*
- backgroundTextureID: int
- animationFrame: int
- animationTimer: float
- totalKills: int
- totalDeaths: int
- timePlayed: float
- levelsCompleted: int

+ GameOverScreen()
+ ~GameOverScreen()
+ update()
+ render()
+ renderFinalScore()
+ renderHighScore()
+ renderStats()
+ renderOptions()
+ selectOption()
+ moveUp()
+ moveDown()
+ confirm()
+ restartGame()
+ goToMainMenu()
+ show()
+ hide()
+ saveHighScore()

MissionSelectScreen

- isVisible: bool
- selectedMission: int
- totalMissions: int
- missionNames: string*
- missionTextureIDs: int*
- isLevelLocked: bool*
- backgroundTextureID: int
- animationFrame: int
- animationTimer: float
- selectedOption: int

+ MissionSelectScreen()
+ ~MissionSelectScreen()
+ update()
+ render()
+ renderMissions()
+ renderMissionDetails()
+ selectMission()
+ moveLeft()
+ moveRight()
+ confirm()
+ goBack()
+ unlockMission()
+ isMissionLocked()
+ show()
+ hide()

CharacterSelectScreen

- isVisible: bool
- selectedCharacter: int
- totalCharacters: int
- characterNames: string*
- characterTextureIDs: int*
- characterDescriptions: string*
- backgroundTextureID: int
- animationFrame: int
- animationTimer: float
- selectedOption: int
- player1Character: int
- player2Character: int

+ CharacterSelectScreen()
+ ~CharacterSelectScreen()
+ update()
+ render()
+ renderCharacters()
+ renderCharacterDetails()
+ renderCharacterStats()
+ selectCharacter()
+ moveLeft()
+ moveRight()
+ confirm()
+ goBack()
+ show()
+ hide()

TimelineScreen

- isVisible: bool
- timelineNodes: TimelineNode**
- nodeCount: int
- selectedNode: int
- currentBranch: int
- totalBranches: int
- scrollX: float
- scrollY: float

+ TimelineScreen()
+ ~TimelineScreen()
+ update()
+ render()
+ renderNodes()
+ renderBranches()
+ selectNode()
+ switchBranch()
+ moveLeft()
+ moveRight()
+ confirm()
+ goBack()
+ show()
+ hide()

ScoreManager

- currentScore: int
- highScore: int
- previousScore: int
- scoreMultiplier: float
- isHighScore: bool
- scoreFileName: string
- player1Score: int
- player2Score: int

+ ScoreManager()
+ ~ScoreManager()
+ update()
+ addScore()
+ subtractScore()
+ resetScore()
+ saveScore()
+ loadScore()
+ getCurrentScore()
+ getHighScore()
+ checkHighScore()
+ updateHighScore()
+ getScoreMultiplier()
+ setScoreMultiplier()
+ renderScore()

KillTracker

- rebelSoldierKills: int
- shieldedSoldierKills: int
- bazookaSoldierKills: int
- grenadeSoldierKills: int
- paratrooperKills: int
- mummyKills: int
- zombieKills: int
- martianKills: int
- ironNokanaKills: int
- hairbusterKills: int
- seaSatanKills: int
- ultimateBossKills: int
- totalKills: int
- meleeKills: int
- aerialKills: int
- multiKills: int
- massacreKills: int
- vehiclesDestroyed: int
- flyingTaraDestroyed: int
- bradleyDestroyed: int
- enemySubDestroyed: int

+ KillTracker()
+ ~KillTracker()
+ trackKill()
+ trackVehicleDestroy()
+ trackMeleeKill()
+ trackAerialKill()
+ trackMultiKill()
+ trackMassacreKill()
+ getTotalKills()
+ getKillsByType()
+ resetKills()
+ checkKillQuota()
+ checkVehicleQuota()
+ getVehiclesDestroyed()

BonusCalculator

- meleeKillBonus: int
- aerialKillBonus: int
- multiKillBonus: int
- massacreBaseBonus: int
- massacrePerKillBonus: int
- survivalClearBonus: int
- campaignClearBonus: int
- flawlessVictoryBonus: int
- lastKillCount: int
- lastKillType: int
- isFlawless: bool

+ BonusCalculator()
+ ~BonusCalculator()
+ calculateMeleeBonus()
+ calculateAerialBonus()
+ calculateMultiKillBonus()
+ calculateMassacreBonus()
+ calculateSurvivalClearBonus()
+ calculateCampaignClearBonus()
+ calculateFlawlessBonus()
+ calculateEnemyKillScore()
+ calculateBossKillScore()
+ checkFlawlessVictory()
+ applyBonus()
+ getTotalBonus()

ScoreBoard

- scores: int*
- scoreCount: int
- maxScores: int
- playerNames: string*
- isVisible: bool
- fileName: string
- topScore: int
- topPlayerName: string

+ ScoreBoard()
+ ~ScoreBoard()
+ update()
+ render()
+ addScore()
+ removeScore()
+ sortScores()
+ saveToFile()
+ loadFromFile()
+ getTopScore()
+ getScoreAt()
+ resetBoard()
+ show()
+ hide()
+ isNewHighScore()

NEATNeuron

- neuronID: int
- neuronType: int
- isInput: bool
- isOutput: bool
- isHidden: bool
- activationValue: float
- bias: float
- activationFunction: int
- layer: int
- x: float
- y: float

+ NEATNeuron()
+ ~NEATNeuron()
+ activate()
+ sigmoid()
+ relu()
+ tanh()
+ reset()
+ getNeuronID()
+ getActivationValue()
+ setActivationValue()
+ getNeuronType()

NEATConnection

- connectionID: int
- innovationNumber: int
- fromNeuronID: int
- toNeuronID: int
- weight: float
- isEnabled: bool
- isRecurrent: bool

+ NEATConnection()
+ ~NEATConnection()
+ enable()
+ disable()
+ mutateWeight()
+ randomizeWeight()
+ getWeight()
+ setWeight()
+ getInnovationNumber()
+ isConnectionEnabled()
+ getFromNeuronID()
+ getToNeuronID()

NEATGenome

- genomeID: int
- neurons: NEATNeuron**
- neuronCount: int
- maxNeurons: int
- connections: NEATConnection**
- connectionCount: int
- maxConnections: int
- fitness: float
- adjustedFitness: float
- speciesID: int
- inputCount: int
- outputCount: int
- hiddenCount: int
- generation: int

+ NEATGenome()
+ ~NEATGenome()
+ addNeuron()
+ removeNeuron()
+ addConnection()
+ removeConnection()
+ mutate()
+ mutateWeights()
+ mutateAddNeuron()
+ mutateAddConnection()
+ mutateToggleConnection()
+ crossover()
+ calculateFitness()
+ getFitness()
+ setFitness()
+ getAdjustedFitness()
+ clone()
+ saveToFile()
+ loadFromFile()
+ getNeuronByID()
+ getConnectionByID()

NEATNetwork

- networkID: int
- genome: NEATGenome*
- inputNeurons: NEATNeuron**
- outputNeurons: NEATNeuron**
- hiddenNeurons: NEATNeuron**
- inputCount: int
- outputCount: int
- hiddenCount: int
- inputValues: float*
- outputValues: float*
- isActive: bool

+ NEATNetwork()
+ ~NEATNetwork()
+ feedForward()
+ setInputs()
+ getOutputs()
+ buildFromGenome()
+ reset()
+ activate()
+ getInputCount()
+ getOutputCount()
+ getOutput()
+ saveToFile()
+ loadFromFile()

NEATSpecies

- speciesID: int
- members: NEATGenome**
- memberCount: int
- maxMembers: int
- representative: NEATGenome*
- bestFitness: float
- averageFitness: float
- adjustedFitness: float
- staleness: int
- maxStaleness: int
- age: int
- compatibilityThreshold: float

+ NEATSpecies()
+ ~NEATSpecies()
+ addMember()
+ removeMember()
+ calculateAverageFitness()
+ calculateAdjustedFitness()
+ updateRepresentative()
+ isCompatible()
+ cull()
+ getBestMember()
+ getSpeciesID()
+ incrementStaleness()
+ isStale()
+ reset()

NEATPopulation

- populationSize: int
- genomes: NEATGenome**
- genomeCount: int
- species: NEATSpecies**
- speciesCount: int
- maxSpecies: int
- generation: int
- bestGenome: NEATGenome*
- bestFitness: float
- averageFitness: float
- innovationNumber: int
- compatibilityThreshold: float
- excessCoefficient: float
- disjointCoefficient: float
- weightCoefficient: float
- mutationRate: float
- crossoverRate: float
- addNeuronRate: float
- addConnectionRate: float

+ NEATPopulation()
+ ~NEATPopulation()
+ initialize()
+ evolve()
+ speciate()
+ reproduce()
+ cull()
+ updateFitness()
+ calculateCompatibility()
+ selectParent()
+ crossover()
+ mutate()
+ getBestGenome()
+ getAverageFitness()
+ incrementInnovation()
+ saveToFile()
+ loadFromFile()
+ resetPopulation()

NEATTrainer

- isActive: bool
- isTraining: bool
- population: NEATPopulation*
- currentGeneration: int
- maxGenerations: int
- isEvolvingIndefinitely: bool
- fitnessThreshold: float
- inputCount: int
- outputCount: int
- gameRef: GameMode*
- playerRef: Character*
- fitnessHistory: float*
- fitnessHistoryCount: int
- saveFilePath: string
- generationTimer: float
- generationDuration: float

+ NEATTrainer()
+ ~NEATTrainer()
+ update()
+ startTraining()
+ stopTraining()
+ runGeneration()
+ evaluateGenome()
+ collectInputs()
+ executeOutputs()
+ calculateFitness()
+ evolvePopulation()
+ saveBestGenome()
+ loadBestGenome()
+ renderTrainingStats()
+ isTrainingComplete()
+ resetTrainer()
+ getInputsFromGame()
+ applyOutputsToPlayer()

GlobalBestPlayer

- isActive: bool
- bestGenome: NEATGenome*
- bestNetwork: NEATNetwork*
- bestFitness: float
- bestGeneration: int
- bestScore: int
- bestWeightsFile: string
- bestTopologyFile: string
- isHighlighted: bool
- highlightColor: int
- highlightAlpha: float
- isBKeyPressed: bool
- isRunning: bool
- sessionBestFitness: float
- allTimeBestFitness: float

+ GlobalBestPlayer()
+ ~GlobalBestPlayer()
+ update()
+ render()
+ activate()
+ deactivate()
+ saveToFile()
+ loadFromFile()
+ updateIfBetter()
+ checkBKeyPressed()
+ highlight()
+ removeHighlight()
+ renderHighlight()
+ getBestFitness()
+ getBestGenome()
+ getBestNetwork()
+ isAllTimeBest()
+ compareWithSession()

DamageProtocol

- isActive: bool
- currentState: int
- previousState: int
- isNormal: bool
- isInjured: bool
- isCritical: bool
- isDead: bool
- injuryTimer: float
- injuryDuration: float
- criticalTimer: float
- criticalDuration: float
- screenRedHue: float
- screenRedHueLight: float
- screenRedHueHeavy: float
- isRedHueActive: bool
- redHueTimer: float
- redHueDuration: float
- hitCount: int
- maxHitsBeforeDeath: int

+ DamageProtocol()
+ ~DamageProtocol()
+ update()
+ render()
+ onHit()
+ transitionToInjured()
+ transitionToCritical()
+ transitionToDead()
+ transitionToNormal()
+ isInInjuredState()
+ isInCriticalState()
+ isInDeadState()
+ activateRedHue()
+ deactivateRedHue()
+ updateRedHue()
+ renderRedHue()
+ resetState()
+ getCurrentState()

PlayerState

- isActive: bool
- currentState: int
- previousState: int
- isNormalState: bool
- isUndeadState: bool
- isMummyState: bool
- undeadTimer: float
- undeadDuration: float
- mummyTimer: float
- mummyDuration: float
- undeadSpeedMultiplier: float
- originalSpeed: float
- mummyWeaponType: int
- originalWeaponType: int
- isWeaponLocked: bool
- isSpeedReduced: bool
- stateTextureID: int
- normalTextureID: int
- undeadTextureID: int
- mummyTextureID: int

+ PlayerState()
+ ~PlayerState()
+ update()
+ render()
+ transitionToUndead()
+ transitionToMummy()
+ transitionToNormal()
+ applyUndeadEffect()
+ removUndeadEffect()
+ applyMummyEffect()
+ removeMummyEffect()
+ lockWeaponToKnife()
+ unlockWeapon()
+ reduceSpeed()
+ restoreSpeed()
+ isUndeadTimerExpired()
+ isMummyTimerExpired()
+ getCurrentState()
+ isInUndeadState()
+ isInMummyState()
+ isInNormalState()

CharacterSwitcher

- isActive: bool
- characters: Character**
- characterCount: int
- currentCharacterIndex: int
- previousCharacterIndex: int
- switchKey: int
- isSwitchable: bool
- character1Inventory: Inventory*
- character2Inventory: Inventory*
- character3Inventory: Inventory*
- character4Inventory: Inventory*
- character1Lives: int
- character2Lives: int
- character3Lives: int
- character4Lives: int
- character1IsDead: bool
- character2IsDead: bool
- character3IsDead: bool
- character4IsDead: bool
- maxLivesPerCharacter: int

+ CharacterSwitcher()
+ ~CharacterSwitcher()
+ update()
+ render()
+ switchToNext()
+ switchTo()
+ checkZKeyPressed()
+ canSwitchToCharacter()
+ getCurrentCharacter()
+ getPreviousCharacter()
+ onCharacterDead()
+ removeDeadCharacter()
+ isAllCharactersDead()
+ getAliveCharacterCount()
+ saveCharacterInventory()
+ loadCharacterInventory()
+ getLivesOfCharacter()
+ decreaseLives()

Inventory

- currentWeapon: Weapon*
- pistol: Pistol*
- knife: Knife*
- equippedWeapon: Weapon*
- grenadeCount: int
- maxGrenades: int
- grenadeType: int
- ammo: int
- maxAmmo: int
- hasPistol: bool
- hasKnife: bool
- hasSpecialWeapon: bool
- specialWeaponType: int
- isPistolActive: bool
- isSpecialWeaponActive: bool
- ownerCharacterType: int

+ Inventory()
+ ~Inventory()
+ equipWeapon()
+ unequipWeapon()
+ pickupWeapon()
+ dropWeapon()
+ addGrenades()
+ removeGrenade()
+ addAmmo()
+ useAmmo()
+ switchToPistol()
+ switchToSpecialWeapon()
+ hasAmmo()
+ hasGrenades()
+ getCurrentWeapon()
+ getGrenadeCount()
+ getAmmo()
+ resetInventory()
+ copyInventory()

SoundManager

- isActive: bool
- isMuted: bool
- masterVolume: float
- musicVolume: float
- sfxVolume: float
- currentMusicID: int
- isMusicPlaying: bool
- isMuffledActive: bool
- muffleStrength: float
- soundEffects: Sound**
- soundEffectCount: int
- maxSoundEffects: int
- backgroundMusic: Sound**
- musicCount: int
- maxMusic: int
- isUnderwater: bool
- audioFFT: AudioFFT*
- currentMusicBuffer: float*
- muffledMusicBuffer: float*
- bufferSize: int

+ SoundManager()
+ ~SoundManager()
+ update()
+ playSound()
+ stopSound()
+ playMusic()
+ stopMusic()
+ pauseMusic()
+ resumeMusic()
+ setMasterVolume()
+ setMusicVolume()
+ setSFXVolume()
+ mute()
+ unmute()
+ activateMuffledMode()
+ deactivateMuffledMode()
+ applyMuffleEffect()
+ loadSound()
+ loadMusic()
+ unloadSound()
+ unloadMusic()
+ getSoundByID()
+ getMusicByID()
+ checkUnderwaterState()
+ updateMuffleState()

Sound

- soundID: int
- name: string
- filePath: string
- volume: float
- isLooping: bool
- isPlaying: bool
- isPaused: bool
- duration: float
- currentTime: float
- isMuffled: bool
- audioBuffer: float*
- bufferSize: int
- sampleRate: int
- channelCount: int

+ Sound()
+ ~Sound()
+ play()
+ stop()
+ pause()
+ resume()
+ setVolume()
+ setLooping()
+ isFinished()
+ getDuration()
+ getCurrentTime()
+ loadFromFile()
+ unload()
+ getAudioBuffer()
+ getBufferSize()

AudioFFT

- isActive: bool
- inputBuffer: float*
- outputBuffer: float*
- frequencyBuffer: float*
- bufferSize: int
- sampleRate: int
- frequencyCount: int
- highFrequencyCutoff: float
- lowFrequencyCutoff: float
- isMuffledOutput: bool
- fftSize: int
- realPart: float*
- imaginaryPart: float*
- magnitudeSpectrum: float*
- phaseSpectrum: float*
- windowFunction: float*
- frequencyResolution: float

+ AudioFFT()
+ ~AudioFFT()
+ performFFT()
+ performInverseFFT()
+ decomposeSoundIntoFrequencies()
+ deleteHighFrequencies()
+ deleteLowFrequencies()
+ reconstructSound()
+ applyWindowFunction()
+ calculateMagnitude()
+ calculatePhase()
+ getMuffledOutput()
+ setHighFrequencyCutoff()
+ setLowFrequencyCutoff()
+ getFrequencyBuffer()
+ getMagnitudeSpectrum()
+ normalizeOutput()
+ loadInputBuffer()
+ clearBuffers()

VoiceDetector

- isActive: bool
- isListening: bool
- microphoneBuffer: float*
- microphoneBufferSize: int
- sampleRate: int
- targetPhrase: string
- targetAudioFile: string
- targetAudioBuffer: float*
- targetAudioSize: int
- targetAudioMatrix: float**
- targetMatrixRows: int
- targetMatrixCols: int
- liveAudioMatrix: float**
- liveMatrixRows: int
- liveMatrixCols: int
- similarityThreshold: float
- cosineSimilarity: float
- isDetected: bool
- audioFFT: AudioFFT*
- detectionTimer: float
- detectionInterval: float
- isPopupActive: bool
- popupMessage: string
- popupTimer: float
- popupDuration: float

+ VoiceDetector()
+ ~VoiceDetector()
+ update()
+ startListening()
+ stopListening()
+ captureMicrophoneInput()
+ loadTargetAudio()
+ decomposeTargetAudio()
+ decomposeLiveAudio()
+ storeTargetInMatrix()
+ storeLiveInMatrix()
+ calculateCosineSimilarity()
+ checkSimilarityThreshold()
+ onPhraseDetected()
+ triggerGameStart()
+ showDetectionPopup()
+ hideDetectionPopup()
+ renderPopup()
+ initMicrophone()
+ closeMicrophone()
+ applyEnvironmentalBlur()
+ render()

EchoChamber

- isActive: bool
- isUnderwater: bool
- audioFFT: AudioFFT*
- inputSound: Sound*
- outputSound: Sound*
- highFrequencyCutoff: float
- muffleStrength: float
- isMuffledActive: bool
- transitionTimer: float
- transitionDuration: float
- isTransitioning: bool

+ EchoChamber()
+ ~EchoChamber()
+ update()
+ activateMuffle()
+ deactivateMuffle()
+ applyMuffleToSound()
+ applyMuffleToMusic()
+ decomposeSoundAtRuntime()
+ deleteHighFrequencies()
+ reconstructMuffledSound()
+ checkUnderwaterState()
+ startTransition()
+ updateTransition()
+ isMuffleActive()

ForksOfFate

- isActive: bool
- isRewinding: bool
- rewindDuration: float
- rewindTimer: float
- currentBranch: int
- totalBranches: int
- currentNode: int
- totalNodes: int
- timelineNodes: TimelineNode**
- timelineBranches: TimelineBranch**
- nodeCount: int
- branchCount: int
- maxNodes: int
- maxBranches: int
- frameBuffer: GameState**
- frameBufferSize: int
- currentFrame: int
- frameRate: float
- saveFilePath: string
- isRKeyPressed: bool

+ ForksOfFate()
+ ~ForksOfFate()
+ update()
+ render()
+ startRewind()
+ stopRewind()
+ rewindFrame()
+ saveFrame()
+ loadFrame()
+ createNode()
+ createBranch()
+ switchBranch()
+ switchNode()
+ saveTimelineToFile()
+ loadTimelineFromFile()
+ checkRKeyPressed()
+ getCurrentNode()
+ getCurrentBranch()
+ renderTimeline()

TimelineBranch

- branchID: int
- parentBranchID: int
- startNodeID: int
- endNodeID: int
- nodes: TimelineNode**
- nodeCount: int
- maxNodes: int
- isActive: bool
- branchTimestamp: float
- saveFilePath: string

+ TimelineBranch()
+ ~TimelineBranch()
+ addNode()
+ removeNode()
+ getNodeAt()
+ saveBranchToFile()
+ loadBranchFromFile()
+ getBranchID()
+ getStartNode()
+ getEndNode()
+ isActiveBranch()

GameState

- playerX: float
- playerY: float
- playerHp: int
- playerAmmo: int
- playerGrenades: int
- playerWeaponType: int
- playerLives: int
- playerScore: int
- playerState: int
- currentCharacter: int
- isInVehicle: bool
- vehicleHp: int
- vehicleType: int
- enemyStates: EnemyState**
- enemyCount: int
- projectileStates: ProjectileState**
- projectileCount: int
- timestamp: float
- frameIndex: int
- currentLevel: int
- isRaining: bool

+ GameState()
+ ~GameState()
+ captureState()
+ restoreState()
+ saveToFile()
+ loadFromFile()
+ getPlayerX()
+ getPlayerY()
+ getTimestamp()

ReflexFingerprint

- isActive: bool
- isRecording: bool
- isPlaying: bool
- playDuration: float
- playTimer: float
- activationKey: int
- movementPatterns: float*
- attackPatterns: int*
- defensePatterns: int*
- weaponSwitchPatterns: int*
- grenadePatterns: float*
- jumpPatterns: float*
- movementPatternCount: int
- attackPatternCount: int
- patternIndex: int
- recordingTimer: float
- samplingRate: float
- fingerprintFile: string
- totalParameterCount: int

+ ReflexFingerprint()
+ ~ReflexFingerprint()
+ update()
+ render()
+ startRecording()
+ stopRecording()
+ startPlaying()
+ stopPlaying()
+ recordMovement()
+ recordAttack()
+ recordDefense()
+ recordWeaponSwitch()
+ recordGrenade()
+ recordJump()
+ replayMovement()
+ replayAttack()
+ replayDefense()
+ saveToFile()
+ loadFromFile()
+ checkActivationKey()
+ isPlayExpired()

FingerprintGhost

- isActive: bool
- x: float
- y: float
- ghostAlpha: float
- ghostColor: int
- hp: int
- maxHp: int
- damage: float
- damageMultiplier: float
- healthMultiplier: float
- fingerprintFile: string
- movementPatterns: float*
- attackPatterns: int*
- defensePatterns: int*
- patternIndex: int
- currentAction: int
- ghostWeaponType: int
- ghostCharacterType: int
- isAutoPlaying: bool

+ FingerprintGhost()
+ ~FingerprintGhost()
+ update()
+ render()
+ loadFingerprintFile()
+ startAutoPlay()
+ stopAutoPlay()
+ replayMovement()
+ replayAttack()
+ replayDefense()
+ applyDamageReduction()
+ applyHealthBoost()
+ renderGhostEffect()
+ isGhostActive()

DeathCompositor

- isActive: bool
- deathFingerprints: DeathFingerprint**
- fingerprintCount: int
- maxFingerprints: int
- currentRunState: DeathFingerprint*
- similarityThreshold: float
- spawnWeightMultiplier: float
- aggressionMultiplier: float
- wallPlacementActive: bool
- isSteeringEnvironment: bool
- saveFilePath: string

+ DeathCompositor()
+ ~DeathCompositor()
+ update()
+ recordDeath()
+ compareWithHistory()
+ checkSimilarity()
+ steerEnvironment()
+ tiltSpawnWeights()
+ increaseAggression()
+ placeDangerousWall()
+ saveToFile()
+ loadFromFile()
+ getMostFrequentDeath()
+ resetCompositor()

DeathFingerprint

- playerHp: int
- playerX: float
- playerY: float
- activeWeapon: int
- enemyConfiguration: int*
- enemyCount: int
- timeSinceCheckpoint: float
- deathCount: int
- frequencyWeight: float
- timestamp: float

+ DeathFingerprint()
+ ~DeathFingerprint()
+ capture()
+ compareWith()
+ saveToFile()
+ loadFromFile()
+ getFrequencyWeight()
+ incrementDeathCount()

RageBait

- isActive: bool
- expectedScore: int
- actualScore: int
- scoreDifference: int
- targetGap: int
- runMidpointScore: int
- historicalMidpointScores: int*
- historicalFinalScores: int*
- historyCount: int
- maxHistory: int
- isManipulating: bool
- lowValueEnemySpawnRate: float
- highValueEnemySpawnRate: float
- saveFilePath: string

+ RageBait()
+ ~RageBait()
+ update()
+ recordMidpointScore()
+ recordFinalScore()
+ calculateExpectedScore()
+ calculateTargetGap()
+ manipulateSpawns()
+ introduceLowValueEnemies()
+ removeHighValueEnemies()
+ saveToFile()
+ loadFromFile()
+ isManipulationActive()
+ getExpectedScore()
+ getScoreDifference()

ChainsOfDeath

- isActive: bool
- currentState: int
- previousState: int
- stateHistory: int*
- stateHistoryCount: int
- observationWindow: float
- observationTimer: float
- movementSpeed: float
- grenadeFrequency: float
- weaponSwapRate: float
- distanceFromEnemies: float
- timeNearCover: float
- timeInOpenSpace: float
- jumpFrequency: float
- meleeFrequency: float
- transitionTable: float**
- stateCount: int
- isSessionLoaded: bool
- recencyWeight: float
- saveFilePath: string
- sessionHistory: float**
- sessionCount: int

+ ChainsOfDeath()
+ ~ChainsOfDeath()
+ update()
+ observePlayer()
+ inferCurrentState()
+ updateTransitionTable()
+ predictNextState()
+ manipulateSpawner()
+ holdHeavyEnemySpawn()
+ accelerateSpawns()
+ introduceDisruption()
+ identifyGapState()
+ forcePlayerIntoGap()
+ updateBossStrategy()
+ saveToFile()
+ loadFromFile()
+ mergeSessionHistory()
+ applyRecencyWeight()
+ getCurrentState()
+ getTransitionProbability()

GrudgefulEnemyManager

- isActive: bool
- passedEnemies: Enemy**
- passedEnemyCount: int
- maxPassedEnemies: int
- counterAttackThreshold: int
- counterAttackTimer: float
- counterAttackInterval: float
- powerBoostMultiplier: float
- isCounterAttackActive: bool
- boostedEnemies: Enemy**
- boostedEnemyCount: int

+ GrudgefulEnemyManager()
+ ~GrudgefulEnemyManager()
+ update()
+ onEnemyPassed()
+ addPassedEnemy()
+ removePassedEnemy()
+ boostPassedEnemies()
+ triggerCounterAttack()
+ checkCounterAttackThreshold()
+ returnEnemiesBack()
+ applyPowerBoost()
+ renderWarningSign()
+ resetManager()

MemoryOfCowardice

- isActive: bool
- ignoredEnemies: Enemy**
- ignoredEnemyCount: int
- maxIgnoredEnemies: int
- reserveCount: int
- counterAttackTimer: float
- counterAttackInterval: float
- isOverwhelmActive: bool
- overwhelmThreshold: int
- overwhelmWarningTimer: float
- overwhelmWarningDuration: float

+ MemoryOfCowardice()
+ ~MemoryOfCowardice()
+ update()
+ render()
+ addIgnoredEnemy()
+ removeIgnoredEnemy()
+ checkOverwhelmThreshold()
+ triggerOverwhelmAttack()
+ releaseReserve()
+ showOverwhelmWarning()
+ resetMemory()
+ getIgnoredEnemyCount()

UrduAssemblyTerminal

- isActive: bool
- isVisible: bool
- inputBuffer: string
- outputBuffer: string
- commandHistory: string*
- historyCount: int
- maxHistory: int
- historyIndex: int
- terminalLines: string*
- lineCount: int
- maxLines: int
- cursorPosition: int
- isCursorVisible: bool
- cursorBlinkTimer: float
- cursorBlinkRate: float
- activationKeyPressCount: int
- activationKeyTimer: float
- activationKeyDuration: float
- activationKeyRequired: int
- backgroundColor: int
- textColor: int
- promptText: string
- interpreter: UrduAssemblyInterpreter*

+ UrduAssemblyTerminal()
+ ~UrduAssemblyTerminal()
+ update()
+ render()
+ open()
+ close()
+ processInput()
+ executeCommand()
+ addToHistory()
+ scrollHistory()
+ clearTerminal()
+ printOutput()
+ printError()
+ checkActivationCombo()
+ renderPrompt()
+ renderCursor()
+ renderLines()
+ handleKeyPress()
+ moveHistoryUp()
+ moveHistoryDown()

UrduAssemblyInterpreter

- isActive: bool
- currentCommand: string
- commandTokens: string*
- tokenCount: int
- maxTokens: int
- variables: string*
- variableValues: int*
- variableCount: int
- maxVariables: int
- gameRef: GameMode*
- playerRef: Character*
- isCommandValid: bool
- lastError: string
- supportedCommands: string*
- supportedCommandCount: int

+ UrduAssemblyInterpreter()
+ ~UrduAssemblyInterpreter()
+ interpret()
+ tokenize()
+ parseCommand()
+ executeJAMA()
+ executeMINUS()
+ validateCommand()
+ validateOperand()
+ modifyHealth()
+ modifyLives()
+ modifyAmmo()
+ modifyGrenades()
+ modifyScore()
+ getVariableValue()
+ setVariableValue()
+ printError()
+ resetInterpreter()

EnemySpawner (Abstract)

# isActive: bool
# spawnList: Enemy**
# spawnCount: int
# maxSpawnCount: int
# spawnTimer: float
# spawnInterval: float
# spawnX: float
# spawnY: float
# spawnBiomeType: int
# spawnLevelType: int
# batchSize: int
# minBatchSize: int
# maxBatchSize: int
# currentBatch: int
# totalBatches: int
# isBatchSpawning: bool
# batchTimer: float
# batchInterval: float
# spawnDirection: int
# isDynamicSpawning: bool
# isFixedSpawning: bool
# fixedPositions: float*
# fixedPositionCount: int
# difficultyMultiplier: float
# enemyTypeWeights: float*
# enemyTypeCount: int

+ EnemySpawner()
+ ~EnemySpawner()
+ update()
+ render()
+ spawnEnemy()
+ spawnBatch()
+ spawnAtPosition()
+ spawnAtFixedPosition()
+ spawnDynamic()
+ setSpawnPosition()
+ setBatchSize()
+ setSpawnInterval()
+ setDifficultyMultiplier()
+ loadFixedPositions()
+ selectEnemyType()
+ canSpawn()
+ isBatchComplete()
+ resetSpawner()
+ pauseSpawning()
+ resumeSpawning()

ParatrooperSpawner

- dropAltitude: float
- dropSpeed: float
- parachuteDeployTimer: float
- parachuteDeployDuration: float
- isParachuteDeployed: bool
- inheritedEnemyType: int

+ ParatrooperSpawner()
+ spawnEnemy()
+ spawnBatch()
+ deployParachute()
+ selectInfantryType()
+ update()
+ render()

BossSpawner

- currentPhase: int
- groundBossSpawned: bool
- aerialBossSpawned: bool
- aquaticBossSpawned: bool
- ultimateBossSpawned: bool
- minionSpawnTimer: float
- minionSpawnInterval: float
- minionBatchSize: int
- minionEnemyType: int
- isMinionSpawning: bool
- crateAfterBatchKill: bool

+ BossSpawner()
+ spawnEnemy()
+ spawnGroundBoss()
+ spawnAerialBoss()
+ spawnAquaticBoss()
+ spawnUltimateBoss()
+ spawnMinions()
+ selectMinionType()
+ checkMinionBatchKilled()
+ spawnCrateAfterBatch()
+ update()
+ render()

SpawnManager

- isActive: bool
- spawners: EnemySpawner**
- spawnerCount: int
- maxSpawners: int
- aerialSpawner: EnemySpawner*
- plainsSpawner: EnemySpawner*
- aquaticSpawner: EnemySpawner*
- infantrySpawner: EnemySpawner*
- undeadSpawner: EnemySpawner*
- alienSpawner: EnemySpawner*
- bossSpawner: EnemySpawner*
- currentLevel: Level*
- currentLevelType: int
- difficultyLevel: int
- totalEnemiesSpawned: int
- totalEnemiesKilled: int
- enemiesKilledPerType: int*
- enemyTypeCount: int
- isDynamicMode: bool
- isFixedMode: bool
- chainsOfDeath: ChainsOfDeath*
- rageBait: RageBait*
- grudgefulManager: GrudgefulEnemyManager*
- memoryOfCowardice: MemoryOfCowardice*
- spawnRadiusAroundPlayer: float
- playerX: float
- playerY: float

+ SpawnManager()
+ ~SpawnManager()
+ update()
+ render()
+ init()
+ initForLevel1()
+ initForLevel2()
+ initForLevel3()
+ initForBossLevel()
+ initForCampaign()
+ addSpawner()
+ removeSpawner()
+ getSpawnerByType()
+ spawnInfantry()
+ spawnAerial()
+ spawnUndead()
+ spawnAliens()
+ spawnBoss()
+ spawnParatroopers()
+ spawnFlyingTara()
+ spawnBradley()
+ spawnEnemySub()
+ spawnZombies()
+ spawnMummies()
+ updatePlayerPosition()
+ adjustSpawnsByChains()
+ adjustSpawnsByRageBait()
+ onEnemyKilled()
+ onEnemyPassed()
+ checkKillQuota()
+ getTotalEnemiesKilled()
+ getEnemiesKilledByType()
+ resetSpawnManager()
+ pauseAllSpawners()
+ resumeAllSpawners()
+ setDifficultyLevel()

Block (Abstract)

# x: float
# y: float
# width: float
# height: float
# blockType: int
# isDestructible: bool
# isIndestructible: bool
# isWater: bool
# isSolid: bool
# hp: int
# biomeType: int
# isBottomLayer: bool
# hasWater: bool
# waterFillTimer: float
# waterFillDuration: float
# craterWidth: float
# craterDepth: float
# isInCrater: bool
# isVisible: bool
# textureID: int
# isExposedToRain: bool
# rainExposureTimer: float

+ Block()
+ ~Block()
+ update()
+ render()
+ takeDamage()
+ destroy()
+ fillWithWater()
+ checkRain()
+ isDestroyed()
+ createCrater()
+ checkCraterSize()
+ checkRainExposure()
+ startWaterFill()
+ isFullyWaterFilled()
+ isSolidBlock()

IndestructibleBlock

- layerIndex: int

+ IndestructibleBlock()
+ takeDamage()
+ destroy()
+ update()
+ render()

***Biome* (Abstract)**

# name: string
# biomeType: int
# startX: float
# endX: float
# startY: float
# endY: float
# seaLevel: float
# minHeight: float
# maxHeight: float
# blocks: Block**
# blockCount: int
# enemySpawner: EnemySpawner*
# isActive: bool
# isUnderwater: bool
# weatherEffect: int
# ambientSoundID: int
# isMuffled: bool
# vehicleSpawnPoints: float*
# vehicleSpawnCount: int
# collectibleSpawnPoints: float*
# collectibleSpawnCount: int
# powPrisonerCount: int

+ Biome()
+ ~Biome()
+ update()
+ render()
+ generateTerrain()
+ spawnEnemies()
+ spawnVehicles()
+ spawnCollectibles()
+ spawnPOWPrisoners()
+ getBlockAt()
+ checkCollision()
+ isInBiome()
+ applyWeather()
+ checkUnderwater()
+ applyAmbientSound()
+ renderBackground()

AerialBiome

- peakHeight: float
- mountainCount: int
- jaggedness: float
- cloudLayer: bool
- windSpeed: float
- windDirection: int

+ AerialBiome()
+ generateTerrain()
+ generateJaggedPeaks()
+ renderClouds()
+ applyWind()
+ update()
+ render()

PlainsBiome

- hillCount: int
- hillHeight: float
- isFlatRegion: bool
- flatRegionStartX: float
- flatRegionEndX: float
- hasVehicleSpawn: bool

+ PlainsBiome()
+ generateTerrain()
+ generateHills()
+ generateFlatRegion()
+ checkIfFlat()
+ update()
+ render()

AquaticBiome

- waterDepth: float
- underwaterEnemyCount: int
- shipCount: int
- underwaterStartY: float
- underwaterEndY: float
- underwaterVisibility: float
- waterCurrentX: float
- waterCurrentY: float
- craterList: Block**
- craterCount: int

+ AquaticBiome()
+ generateTerrain()
+ generateSeaFloor()
+ applyUnderwaterEffects()
+ applyWaterCurrent()
+ activateMuffledSound()
+ deactivateMuffledSound()
+ updateCraters()
+ update()
+ render()
+ renderWaterSurface()
+ renderUnderwaterEffect()

TimelineNode

- nodeID: int
- branchID: int
- frameIndex: int
- gameState: GameState*
- timestamp: float
- isActive: bool
- parentNodeID: int
- childNodeIDs: int*
- childCount: int
- positionX: float
- positionY: float

+ TimelineNode()
+ ~TimelineNode()
+ saveState()
+ loadState()
+ addChild()
+ removeChild()
+ getNodeID()
+ getBranchID()
+ getGameState()
+ isRootNode()

PerlinNoise

- seed: int
- frequency: float
- amplitude: float
- permutationTable: int*
- tableSize: int

+ PerlinNoise()
+ ~PerlinNoise()
+ generate()
+ fade()
+ lerp()
+ grad()
+ shuffle()
+ setSeed()

FractalNoise

- octaves: int
- persistence: float
- lacunarity: float
- scale: float
- noiseLayers: PerlinNoise**
- layerCount: int

+ FractalNoise()
+ ~FractalNoise()
+ generate()
+ addLayer()
+ removeLayer()
+ setOctaves()
+ getHeightMap()

NoiseProfileFactory

- amplifiedProfile: AmplifiedProfile*
- flatProfile: FlatProfile*
- normalProfile: NormalProfile*

+ NoiseProfileFactory()
+ ~NoiseProfileFactory()
+ createProfile()
+ createAmplifiedProfile()
+ createFlatProfile()
+ createNormalProfile()
+ getProfile()

Rain

- isActive: bool
- rainTimer: float
- rainDuration: float
- rainInterval: float
- rainIntensity: float
- rainDropCount: int
- rainDropX: float*
- rainDropY: float*
- rainDropSpeed: float
- rainDropLength: float
- isMuffled: bool

+ Rain()
+ ~Rain()
+ start()
+ stop()
+ update()
+ render()
+ isRaining()
+ getRainIntensity()
+ checkCraterFill()

CraterWaterFill

- craterBlock: Block*
- isFillingActive: bool
- fillTimer: float
- fillDuration: float
- isFullyFilled: bool
- craterWidth: float
- craterDepth: float
- maxCraterWidth: float
- minCraterDepth: float

+ CraterWaterFill()
+ ~CraterWaterFill()
+ startFilling()
+ update()
+ render()
+ checkCraterSize()
+ isFilled()
+ isExposedToRain()

EnvironmentManager

- rain: Rain*
- craterFills: CraterWaterFill**
- craterFillCount: int
- isRaining: bool
- rainCheckTimer: float
- rainCheckInterval: float
- currentLevel: Level*

+ EnvironmentManager()
+ ~EnvironmentManager()
+ update()
+ render()
+ startRain()
+ stopRain()
+ checkRandomRain()
+ updateCraterFills()
+ addCraterFill()
+ removeCraterFill()
+ onBlastCreatesCrater()

***NoiseProfile* (Abstract)**

# name: string
# profileType: int
# peakAmplitude: float
# oceanDepth: float
# plainsScale: float
# frequency: float
# persistence: float
# lacunarity: float
# octaves: int

+ NoiseProfile()
+ ~NoiseProfile()
+ applyProfile()
+ getAmplitude()
+ getFrequency()
+ getPersistence()
+ getLacunarity()
+ getOctaves()

AmplifiedProfile

- peakMultiplier: float
- oceanMultiplier: float

+ AmplifiedProfile()
+ applyProfile()
+ getAmplitude()

FlatProfile

- flatnessFactor: float

+ FlatProfile()
+ applyProfile()
+ getAmplitude()

NormalProfile

- distributionFactor: float

+ NormalProfile()
+ applyProfile()
+ getAmplitude()

NOTE: All the classes that dont have the ABSTRACT tag are CONCRETE.